

Name/NetID: Nathan Kariuki / nkari011

Session Date: May 12, 2026 (1 hour)

Objective: Implement the ConcurrentQueue and Graph types.

Attempts Made:

Phase 3 adds a live update pipeline on top of the static graph work from earlier phases. A single producer reads from a trace file and pushes ADD requests into a shared queue; N consumer threads dequeue and apply them to an adjacency list. Before wiring anything together I wanted to get the two core data structures solid first.

ConcurrentQueue needs Push, Pop, and Close. The termination logic is the main thing to get right. My first attempt had Pop wait on !q.empty(), but that deadlocks — if the producer calls Close() while a consumer is already parked on an empty queue, nothing ever wakes it up. The fix is to wait on !q.empty() || closed, so a consumer wakes on either a new item or the queue closing. Push uses notify_one since only one consumer needs to wake per item; Close uses notify_all because every parked consumer needs to observe the closed state independently and return nullopt.

For the Graph, each source vertex gets its own std::mutex in a parallel locks[] vector. Applying an ADD locks only locks[src], does a push_back, then unlocks. Two consumers updating different rows never contend; two hitting the same src will serialize, but that's unavoidable and correct.

Solutions/Workarounds:

The notify_one vs notify_all distinction was the only real design question. Once that was clear the code followed straightforwardly. First commit pushed with concurrent_queue.h and graph.h.

Learning Outcome:

A condition variable predicate needs to cover every reason a thread should stop waiting, not just the happy path. Missing the closed condition causes a permanent deadlock which is quite easy to miss if the queue rarely drains fully.